Apuntes PSP – Hilos

ndice

Índice rápido

Busca con Ctrl+F la clave entre corchetes para saltar a la sección.

Clave	Qué necesitas hacer	Sección
`[TEORIA]`	Conceptos, estados, métodos de Thread	1
`[CREAR]`	Crear y arrancar hilos (Thread, Runnable, lambda)	2
`[ESTADOS]`	Ver los estados de un hilo con getState	3
`[CONTROL]`	sleep, interrupt, join, yield, prioridad, nombre	4
`[TIMER]`	Ejecutar algo cada X tiempo con Timer	5
`[SYNC]`	Contador compartido sin errores: synchronized	6
`[DEADLOCK]`	Ejemplo de interbloqueo y cómo evitarlo	7
`[PRODCON]`	Productor-consumidor con wait / notifyAll	8
`[SEMAFORO]`	Productor-consumidor con Semaphore	9
`[EXCEPCION]`	Capturar excepciones de todos los hilos en un sitio	10
`[EXECUTOR]`	Pool de hilos, tareas programadas	11
`[EXCHANGER]`	Intercambiar datos entre dos hilos	12
`[ERRORES]`	Fallos típicos de examen	13

Regla general: el código que hace el hilo va en `run()`, pero se arranca con `start()`. Si llamas a `run()` directamente NO se crea un hilo nuevo, se ejecuta en el hilo actual como un método normal.

1. Teoría [TEORIA]

Hilo vs proceso

- Todo proceso tiene al menos un hilo (el `main`).
- Los hilos de un mismo proceso **comparten memoria** (variables, objetos): por eso pueden pisarse y hace falta sincronizar. Los procesos NO comparten memoria.
- Crear un hilo es mucho más ligero que crear un proceso. Con `ps -ef` solo verás un proceso Java aunque tenga muchos hilos.
- **Secuencial:** una instrucción detrás de otra. **Concurrente:** varios hilos avanzan a la vez (intercalados o en paralelo).

start() vs run()

- `start()`: crea un hilo nuevo y ese hilo ejecuta `run()`. Es lo que hay que usar.
- `run()` llamado a mano: se ejecuta en el hilo actual, sin concurrencia.
- Un mismo objeto Thread solo puede hacer `start()` una vez (si no: `IllegalThreadStateException`).

Thread vs Runnable

- `extends Thread`: más directo, pero ya no puedes heredar de otra clase.
- `implements Runnable`: más flexible; un mismo objeto Runnable puede compartirse entre varios hilos (comparten sus atributos).

Estados de un hilo (Thread.State)

Estado	Java	Cuándo
Nuevo	`NEW`	Creado, aún sin `start()`
Ejecutable	`RUNNABLE`	Ejecutándose o listo para ejecutarse
Bloqueado	`BLOCKED`	Esperando entrar en un `synchronized` ocupado
Esperando	`WAITING`	En `wait()` o `join()` sin tiempo
Esperando tiempo	`TIMED_WAITING`	En `sleep(ms)`, `wait(ms)`, `join(ms)`
Finalizado	`TERMINATED`	`run()` ha acabado

Métodos de Thread

Método	Qué hace
`start()`	Arranca el hilo (llama a `run()` en un hilo nuevo)
`run()`	Código del hilo
`Thread.sleep(ms)`	Duerme el hilo actual (lanza `InterruptedException`)
`interrupt()`	Avisa al hilo de que pare; si duerme/espera

Método	Qué hace
	salta `InterruptedException`
`isInterrupted()`	¿Le han interrumpido?
`join()` / `join(ms)`	Espera a que ese hilo termine
`Thread.yield()`	Cede la CPU voluntariamente (≈ `Thread.sleep(0)`)
`isAlive()`	¿Sigue vivo?
`setName()` / `getName()`	Nombre del hilo
`setPriority(n)`	1 a 10: `MIN_PRIORITY`=1, `NORM_PRIORITY`=5, `MAX_PRIORITY`=10
`getState()`	Estado actual
`Thread.currentThread()`	El hilo que está ejecutando esta línea
`setDaemon(true)`	Hilo demonio: no impide que el programa termine

Conceptos de sincronización

- **Recurso compartido / monitor:** objeto al que acceden varios hilos; en Java cada objeto tiene un cerrojo (monitor).
- **Sección crítica:** código que toca el recurso compartido.
- **Exclusión mutua:** solo un hilo a la vez en la sección crítica (`synchronized`, `Semaphore`).
- **Operación atómica:** no se puede interrumpir a medias. `contador++` NO es atómica (leer, sumar, escribir).
- **Interbloqueo (deadlock):** dos hilos se esperan mutuamente y se quedan parados para siempre.
- **Inanición:** un hilo nunca consigue el recurso.
- **Herramientas:** `join`, `synchronized`, `wait/notify/notifyAll`, `Semaphore` (`acquire`, `release`, `availablePermits`, `isFair`).

2. Crear y arrancar hilos [CREAR]

Opción A: heredar de Thread

```
public class MiHilo extends Thread {
    private String nombre;

    public MiHilo(String nombre) {
        super(nombre);           // pone el nombre del hilo
(getName())
        this.nombre = nombre;
    }

    @Override
    public void run() {
        for (int i = 0; i < 5; i++) {
```

```
            System.out.println(nombre + " vuelta " + i);
        }
    }

    public static void main(String[] args) {
        MiHilo h1 = new MiHilo("Hilo 1");
        MiHilo h2 = new MiHilo("Hilo 2");
        h1.start();
        h2.start();
    }
}
```

Opción B: implementar Runnable

```
public class MiTarea implements Runnable {
    @Override
    public void run() {
        System.out.println("Soy " + Thread.currentThread().getName());
    }

    public static void main(String[] args) {
        MiTarea tarea = new MiTarea();
        Thread t1 = new Thread(tarea, "Hilo 1");   // (Runnable,
nombre)
        Thread t2 = new Thread(tarea, "Hilo 2");
        t1.start();
        t2.start();
    }
}
```

Opción C: lambda (rápido para el examen)

```
Thread t = new Thread(() -> {
    System.out.println("Hola desde " +
Thread.currentThread().getName());
});
t.start();
```

Muchos hilos compartiendo un Runnable (ejemplo "ratones")

Todos los hilos comparten el mismo objeto, así que comparten `iValor`. Con muchos hilos y varios núcleos, el resultado puede ser menor de 10000 porque `iValor++` no es atómico → hace falta exclusión mutua (sección 6).

```
public class Ratones implements Runnable {
    int iValor = 0;

    @Override
    public void run() {
        iValor++;                          // sección crítica SIN
proteger
    }
```

```
    public static void main(String[] args) throws InterruptedException
{
        Ratones r = new Ratones();
        Thread[] hilos = new Thread[10000];
        for (int i = 0; i < hilos.length; i++) {
            hilos[i] = new Thread(r);
            hilos[i].start();
        }
        for (Thread h : hilos) h.join();  // esperar a todos (mejor
que sleep)
        System.out.println(r.iValor);
    }
}
```

Pasar datos al hilo y recoger resultados

```
public class Sumador extends Thread {
    private int desde, hasta;
    private long resultado;              // el main lo lee DESPUES del
join

    public Sumador(int desde, int hasta) { this.desde = desde;
this.hasta = hasta; }

    @Override
    public void run() {
        for (int i = desde; i <= hasta; i++) resultado += i;
    }

    public long getResultado() { return resultado; }

    public static void main(String[] args) throws InterruptedException
{
        Sumador s1 = new Sumador(1, 500);
        Sumador s2 = new Sumador(501, 1000);
        s1.start(); s2.start();
        s1.join();  s2.join();
        System.out.println("Total: " + (s1.getResultado() +
s2.getResultado()));
    }
}
```

3. Estados de un hilo [ESTADOS]

Guarda en una lista cada estado distinto por el que pasa el hilo. Salida típica: `NEW`, `RUNNABLE`, `TIMED_WAITING`, `TERMINATED`.

```
import java.util.ArrayList;

public class EstadosHilo implements Runnable {
```

```
    @Override
    public void run() {
        try {
            Thread.sleep(100);           // provoca el estado
TIMED_WAITING
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
    }

    public static void main(String[] args) {
        ArrayList<Thread.State> estados = new ArrayList<>();
        Thread hilo = new Thread(new EstadosHilo());

        estados.add(hilo.getState());    // NEW
        hilo.start();

        while (hilo.getState() != Thread.State.TERMINATED) {
            if (!estados.contains(hilo.getState())) {
                estados.add(hilo.getState());
            }
        }
        estados.add(hilo.getState());    // TERMINATED

        for (Thread.State e : estados) {
            System.out.println(e);
        }
    }
}
```

Para ver `WAITING` usa `wait()` o `join()` sin tiempo dentro del hilo; para `BLOCKED`, dos hilos intentando entrar en el mismo `synchronized`.

4. sleep, interrupt, join, yield, prioridad [CONTROL]

join: el main espera a los hijos

Con `join` el mensaje "Termina programa" sale siempre al final. Sin `join`, sale entre medias de los mensajes de los hijos.

```
public class JoinYield {
    public static void main(String[] args) {
        Hijo h1 = new Hijo("Hijo uno", true);    // este hace yield en
la vuelta 5
        Hijo h2 = new Hijo("Hijo dos", false);
        h1.start();
        h2.start();
        try {
            h1.join();
```

```
            h2.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("Termina programa");
    }
}

class Hijo extends Thread {
    private String nombre;
    private boolean cede;

    public Hijo(String nombre, boolean cede) {
        this.nombre = nombre;
        this.cede = cede;
    }

    @Override
    public void run() {
        System.out.println("Iniciando " + nombre);
        for (int i = 0; i < 500; i++) {
            if (cede && i == 5) {
                Thread.yield();      // cede la CPU (equivale a
Thread.sleep(0))
            }
            System.out.println(nombre + " Pasada: " + i);
        }
        System.out.println("Terminando " + nombre);
    }
}
```

`yield` solo es una sugerencia: el planificador puede dar la CPU al otro hijo, al padre, o volver a darla al mismo hilo.

interrupt: parar un hilo con bucle infinito

```
public class Trabajador implements Runnable {
    @Override
    public void run() {
        while (!Thread.currentThread().isInterrupted()) {
            System.out.println("Trabajando...");
            try {
                Thread.sleep(1000);
            } catch (InterruptedException e) {
                break;               // nos han interrumpido mientras
dormia -> salir
            }
        }
        System.out.println("Hilo terminado");
    }
```

```
    public static void main(String[] args) throws InterruptedException
{
        Thread t = new Thread(new Trabajador());
        t.start();
        Thread.sleep(5000);          // dejarlo trabajar 5 s
        t.interrupt();               // pedirle que pare
        t.join();
        System.out.println("Main termina");
    }
}
```

Nombre y prioridad

```
Thread t = new Thread(tarea);
t.setName("Trabajador-1");
t.setPriority(Thread.MAX_PRIORITY);   // 10   (MIN_PRIORITY = 1,
NORM_PRIORITY = 5)
t.start();
System.out.println(t.getName() + " prioridad " + t.getPriority() + "
vivo? " + t.isAlive());
```

La prioridad es una pista para el planificador, no garantiza el orden.

5. Timer y TimerTask [TIMER]

Un `Timer` ejecuta una `TimerTask` en un hilo propio (con `ps -ef` se ve que es un hilo, no un proceso nuevo). Si no haces `cancel()`, el timer sigue vivo aunque `main` termine.

```
import java.util.Timer;

public class EjemploTimer {
    public static void main(String[] args) {
        Timer timer = new Timer();

        // schedule(tarea, retrasoInicial_ms)              -> se
ejecuta UNA vez
        // schedule(tarea, retrasoInicial_ms, periodo_ms)  -> se
repite
        timer.schedule(new TareaHora(), 1000, 2000);   // empieza a 1
s, repite cada 2 s

        try { Thread.sleep(6000); } catch (InterruptedException e)
{ e.printStackTrace(); }

        timer.cancel();                                 // parar el
timer
        System.out.println("Parado el timer");

        try { Thread.sleep(5000); } catch (InterruptedException e)
{ e.printStackTrace(); }
        System.out.println("Finaliza el programa");
```

```
    }
}

import java.text.SimpleDateFormat;
import java.util.Date;
import java.util.TimerTask;

public class TareaHora extends TimerTask {
    @Override
    public void run() {
        System.out.println(new SimpleDateFormat("HH:mm:ss").format(new
Date()));
    }
}
```

Tarea simple de ejemplo:

```
import java.util.TimerTask;

public class TareaSaludo extends TimerTask {
    @Override
    public void run() {
        System.out.println("Saludos 2-DAM");
    }
}
```

Otras opciones: `timer.scheduleAtFixedRate(tarea, 0, 1000)` (ritmo fijo aunque una ejecución se retrase) y `tarea.cancel()` para cancelar solo esa tarea.

6. synchronized: exclusión mutua con monitor [SYNC]

Varios hilos incrementan el mismo contador. **Sin** `synchronized` el total sale menor de lo esperado (se pierden incrementos); **con** `synchronized` sale exacto (6 hilos × 600000 = 3.600.000). Se nota mejor con varios núcleos (en VirtualBox: Configuración → Sistema → Procesador).

Recurso compartido (monitor) con método synchronized

`synchronized` en un método bloquea el objeto entero (`this`) mientras un hilo está dentro.

```
public class Contador {
    private int iContador = 0;

    public synchronized void incrementa() {     // quitar synchronized
para ver el fallo
        iContador = iContador + 1;
    }

    public synchronized int getContador() {
        return iContador;
```

```
    }
}
```

Hilo que usa el recurso

```
public class HiloContador extends Thread {
    private Contador zonaComp;
    private String sNombre;

    public HiloContador(Contador zonaComp, String sNombre) {
        this.zonaComp = zonaComp;
        this.sNombre = sNombre;
    }

    @Override
    public void run() {
        for (int i = 0; i < 600000; i++) {
            zonaComp.incrementa();
            // System.out.println("Soy " + sNombre + ": " +
zonaComp.getContador()); // ralentiza mucho
        }
    }
}
```

Main

```
public class MainMonitor {
    public static void main(String[] args) {
        Contador monitor = new Contador();          // UN solo objeto
compartido por todos
        HiloContador[] hilos = new HiloContador[6];
        for (int i = 0; i < hilos.length; i++) {
            hilos[i] = new HiloContador(monitor, "Hilo " + (i + 1));
            hilos[i].start();
        }
        try {
            for (HiloContador h : hilos) h.join();
        } catch (InterruptedException e) {
            e.printStackTrace();
        }
        System.out.println("Total: " + monitor.getContador());   //
3600000
    }
}
```

Variante: synchronized por bloque

El método `incrementa()` del recurso se deja SIN synchronized y se protege desde el hilo. Útil para hacer atómicas varias operaciones juntas (incrementar + imprimir).

```
@Override
public void run() {
    for (int i = 0; i < 200000; i++) {
```

```
        synchronized (zonaComp) {                    // cerrojo sobre
el objeto compartido
            zonaComp.incrementa();
            System.out.println("Soy " + sNombre + ": " +
zonaComp.getContador());
        }
    }
}
```

Variante: cerrojo estático (cuando cada hilo es un objeto distinto)

```
class Hilo extends Thread {
    static int contador = 0;                          // compartido por
ser static
    static final Object cerrojo = new Object();

    public void run() {
        for (int i = 0; i < 1000; i++) {
            synchronized (cerrojo) { contador++; }
        }
    }
}
```

Ojo: `synchronized` en un método de una clase que hereda de Thread NO sirve si cada hilo es un objeto distinto: cada uno bloquea su propio `this`. El cerrojo tiene que ser el objeto COMPARTIDO.

7. Interbloqueo (deadlock) [DEADLOCK]

Hilo1 bloquea ArchivoA y luego quiere ArchivoB; Hilo2 bloquea ArchivoB y luego quiere ArchivoA. Cada uno tiene lo que el otro necesita → los dos se quedan parados para siempre (estado `BLOCKED`).

```
public class MainDeadlock {
    static ArchivoA a = new ArchivoA();
    static ArchivoB b = new ArchivoB();

    public static void main(String[] args) {
        new Hilo1(a, b).start();
        new Hilo2(a, b).start();
    }
}

class Hilo1 extends Thread {
    ArchivoA a; ArchivoB b;
    Hilo1(ArchivoA a, ArchivoB b) { this.a = a; this.b = b; }
    @Override
    public void run() { a.metodoA(b); }       // captura ArchivoA
primero
}
```

```
class Hilo2 extends Thread {
    ArchivoA a; ArchivoB b;
    Hilo2(ArchivoA a, ArchivoB b) { this.a = a; this.b = b; }
    @Override
    public void run() { b.metodoB(a); }       // captura ArchivoB
primero
}

class ArchivoA {
    public synchronized void metodoA(ArchivoB b) {
        String name = Thread.currentThread().getName();
        System.out.println(name + " entra en ArchivoA");
        try { Thread.sleep(1000); } catch (InterruptedException e) { }
        System.out.println(name + " intentando acceder a ArchivoB");
        b.metodoB(this);                        // necesita el cerrojo
de B
    }
}

class ArchivoB {
    public synchronized void metodoB(ArchivoA a) {
        String name = Thread.currentThread().getName();
        System.out.println(name + " entra en ArchivoB");
        try { Thread.sleep(1000); } catch (InterruptedException e) { }
        System.out.println(name + " intentando acceder a ArchivoA");
        a.metodoA(this);                        // necesita el cerrojo
de A
    }
}
```

Cómo evitarlo

- **Mismo orden de cerrojos:** todos los hilos piden siempre A y después B.
- No llamar a otro método `synchronized` de otro objeto desde dentro de uno `synchronized`.
- Usar tiempos de espera (`tryLock` con `ReentrantLock`, `tryAcquire` con `Semaphore`).

```
// Version sin deadlock: los dos hilos bloquean en el mismo orden
synchronized (a) {
    synchronized (b) {
        // usar los dos recursos
    }
}
```

8. Productor-consumidor con wait / notifyAll [PRODCON]

El monitor tiene un almacén de capacidad REG. El productor espera (`wait`) si está lleno; el consumidor espera si está vacío. Tras cada cambio se avisa a todos con `notifyAll()`. El main los para a los 20 s con `interrupt()`.

Reglas: `wait()` y `notifyAll()` solo dentro de `synchronized`; la condición siempre con `while`, nunca con `if`.

Monitor

```
import java.time.LocalTime;
import java.util.LinkedList;

public class Monitor {
    private final int REG = 3;                          // capacidad
del almacen
    private LinkedList<String> almacen = new LinkedList<>();

    public synchronized void producir(String quien) throws
InterruptedException {
        while (almacen.size() == REG) {                 // lleno ->
esperar
            wait();
        }
        String dato = LocalTime.now().withNano(0).toString();
        almacen.addLast(dato);
        System.out.println(quien + " produce " + dato + " (hay " +
almacen.size() + ")");
        notifyAll();                                    // despertar a
consumidores
    }

    public synchronized String consumir(String quien) throws
InterruptedException {
        while (almacen.isEmpty()) {                     // vacio ->
esperar
            wait();
        }
        String dato = almacen.removeFirst();
        System.out.println(quien + " consume " + dato + " (hay " +
almacen.size() + ")");
        notifyAll();                                    // despertar a
productores
        return dato;
    }
}
```

(En los apuntes de clase la versión 1 usa un array `String[REG]` con el índice `iNumAlm`, y la versión 2 un `TreeSet` con un `Iterator`; la lógica de `while`/`wait`/`notifyAll` es la misma.)

Productor

```
public class Productor implements Runnable {
    private Monitor miMon;
    private String sNom;

    public Productor(Monitor mon, String nombre) { miMon = mon; sNom =
```

```
nombre; }

    @Override
    public void run() {
        try {
            while (true) {                                  // bucle
infinito, sale con interrupt
                miMon.producir(sNom);
                Thread.sleep(3000 + (int) (Math.random() * 5000));
            }
        } catch (InterruptedException e) {
            // interrumpido -> terminar
        }
        System.out.println("Terminando productor " + sNom);
    }
}
```

Consumidor

```
public class Consumidor implements Runnable {
    private Monitor miMon;
    private String sNom;

    public Consumidor(Monitor mon, String nombre) { miMon = mon; sNom
= nombre; }

    @Override
    public void run() {
        try {
            while (true) {
                miMon.consumir(sNom);
                Thread.sleep(3000 + (int) (Math.random() * 5000));
            }
        } catch (InterruptedException e) {
            // interrumpido -> terminar
        }
        System.out.println("Terminando consumidor " + sNom);
    }
}
```

Main

```
public class MainProdCon {
    public static void main(String[] args) {
        Monitor miMon = new Monitor();

        Thread p1 = new Thread(new Productor(miMon, "PROD-1"));
        Thread p2 = new Thread(new Productor(miMon, "PROD-2"));
        Thread c1 = new Thread(new Consumidor(miMon, "CON-1"));
        Thread c2 = new Thread(new Consumidor(miMon, "CON-2"));

        p1.start(); c1.start(); p2.start(); c2.start();
```

```
        try { Thread.sleep(20000); } catch (InterruptedException e)
{ e.printStackTrace(); }

        p1.interrupt(); c1.interrupt(); p2.interrupt();
c2.interrupt();
    }
}
```

Adaptar: cambia el tipo del almacén (`LinkedList<Integer>`, objetos propios…), `REG`, lo que se produce y los tiempos de `sleep`. `notify()` despierta a uno solo; `notifyAll()` a todos (más seguro con varios productores/consumidores).

9. Productor-consumidor con Semaphore [SEMAFORO]

Un `Semaphore` tiene N permisos. `acquire()` coge uno (si no hay, el hilo espera); `release()` lo devuelve. Con `new Semaphore(1)` funciona como un cerrojo (exclusión mutua).

Método	Qué hace
`new Semaphore(n, true)`	n permisos; `true` = justo (FIFO, evita inanición)
`acquire()`	Coge un permiso o espera (lanza `InterruptedException`)
`release()`	Devuelve un permiso
`tryAcquire()`	Intenta coger sin esperar, devuelve `boolean`
`availablePermits()`	Permisos libres ahora
`isFair()`	¿Es justo?

Versión de clase (Monitor con semáforo)

Mismos `Productor`, `Consumidor` y `Main` que en la sección 8 (aquí `Producir()`/`Consumir()` no lanzan excepción ni reciben nombre: ajusta la llamada). El consumidor comprueba si hay algo antes de sacar.

```
import java.time.LocalDateTime;
import java.util.Iterator;
import java.util.Set;
import java.util.TreeSet;
import java.util.concurrent.Semaphore;

public class Monitor {
    final int REG = 3;
    Semaphore almacen = new Semaphore(REG, true);
    int iNumAlm = 0;
    Set<String> miCol = new TreeSet<String>();

    public synchronized void Producir() {
        try { almacen.acquire(); } catch (InterruptedException e)
```

```
{ e.printStackTrace(); }
        LocalDateTime ldt = LocalDateTime.now();
        String sTiempo = ldt.getHour() + ":" + ldt.getMinute() + ":" +
ldt.getSecond();
        miCol.add(sTiempo);
        System.out.println("Produce a las " + sTiempo);
        iNumAlm++;
        almacen.release();
    }

    public synchronized void Consumir() {
        try { almacen.acquire(); } catch (InterruptedException e)
{ e.printStackTrace(); }
        if (!miCol.isEmpty()) {
            iNumAlm--;
            Iterator<String> miIter = miCol.iterator();
            System.out.println("Consume a las " + miIter.next());
            miIter.remove();
        }
        almacen.release();
    }
}
```

Versión clásica con tres semáforos (respeta la capacidad y espera de verdad)

- `huecos` (empieza en REG): cuántos sitios libres quedan → el productor espera si es 0.
- `llenos` (empieza en 0): cuántos elementos hay → el consumidor espera si es 0.
- `mutex` (1): exclusión mutua al tocar la lista. **No** se usa `synchronized`.

```
import java.util.LinkedList;
import java.util.concurrent.Semaphore;

public class MonitorSemaforos {
    private final int REG = 3;
    private LinkedList<String> almacen = new LinkedList<>();
    private Semaphore huecos = new Semaphore(REG, true);
    private Semaphore llenos = new Semaphore(0, true);
    private Semaphore mutex  = new Semaphore(1, true);

    public void producir(String quien, String dato) throws
InterruptedException {
        huecos.acquire();                // esperar hueco libre
        mutex.acquire();
        almacen.addLast(dato);
        System.out.println(quien + " produce " + dato + " (hay " +
almacen.size() + ")");
        mutex.release();
        llenos.release();                // hay un elemento mas
    }

    public String consumir(String quien) throws InterruptedException {
```

```
        llenos.acquire();                   // esperar a que haya algo
        mutex.acquire();
        String dato = almacen.removeFirst();
        System.out.println(quien + " consume " + dato + " (hay " +
almacen.size() + ")");
        mutex.release();
        huecos.release();                   // queda un hueco libre
        return dato;
    }
}
```

Semáforo para limitar cuántos hilos entran a la vez

```
Semaphore sem = new Semaphore(2, true);       // solo 2 hilos a la vez
(ej. 2 cajeros)

Runnable cliente = () -> {
    try {
        sem.acquire();
        System.out.println(Thread.currentThread().getName() + "
atendido. Libres: " + sem.availablePermits());
        Thread.sleep(2000);
    } catch (InterruptedException e) {
        e.printStackTrace();
    } finally {
        sem.release();                          // en finally para no
perder el permiso
    }
};
for (int i = 1; i <= 5; i++) new Thread(cliente, "Cliente " +
i).start();
```

10. Excepciones centralizadas: UncaughtExceptionHandler [EXCEPCION]

Una excepción no capturada dentro de `run()` mata solo ese hilo y no llega al `main`. Con un manejador común se recogen las de todos los hilos en un solo sitio.

```
// Manejador comun
public class ManejadorExcepciones implements
Thread.UncaughtExceptionHandler {
    @Override
    public void uncaughtException(Thread hilo, Throwable excepcion) {
        System.out.println("El hilo '" + hilo.getName() + "' ha
devuelto la excepcion:");
        excepcion.printStackTrace();
    }
}

import java.util.Random;

// Hilo que a veces divide entre 0
public class Hilo extends Thread {
```

```
    public Hilo(String nombre) { super(nombre); }

    @Override
    public void run() {
        int division = 100 / new Random().nextInt(4);   // 0..3 ->
ArithmeticException si sale 0
        System.out.println(getName() + " Division: " + division);
    }
}

public class MainExcepciones {
    public static void main(String[] args) {
        ManejadorExcepciones manejador = new ManejadorExcepciones();
        for (int i = 1; i <= 5; i++) {
            Hilo h = new Hilo("hilo" + i);
            h.setUncaughtExceptionHandler(manejador);   // asociar a
cada hilo
            h.start();
        }
        // Alternativa para TODOS los hilos de golpe:
        // Thread.setDefaultUncaughtExceptionHandler(manejador);
    }
}
```

11. Executors: pool de hilos y tareas programadas [EXECUTOR]

Alternativa a `Timer` y a crear hilos a mano (paquete `java.util.concurrent`). Las tareas son `Runnable`.

Tarea repetida (sustituye a Timer)

```
import java.util.concurrent.Executors;
import java.util.concurrent.ScheduledExecutorService;
import java.util.concurrent.TimeUnit;

ScheduledExecutorService ses =
Executors.newSingleThreadScheduledExecutor();
ses.scheduleAtFixedRate(new TareaUno(), 1, 2, TimeUnit.SECONDS); //
empieza a 1 s, cada 2 s
// ses.schedule(new TareaUno(), 3, TimeUnit.SECONDS);           //
una sola vez a los 3 s

Thread.sleep(10000);   // dentro de try/catch
ses.shutdown();        // parar (si no, el programa no termina)
```

Pool de tamaño fijo (cola de tareas)

Solo se ejecutan 3 tareas a la vez; el resto espera en cola. Si una tarea no termina nunca (`TareaDos`), ocupa su hilo para siempre y las siguientes no arrancan.

```
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
```

```java
import java.util.concurrent.TimeUnit;

public class EjemploExecutor {
    public static void main(String[] args) throws InterruptedException
{
        ExecutorService pool = Executors.newFixedThreadPool(3);

        pool.submit(new TareaUno());
        pool.submit(new TareaDos());
        pool.submit(new TareaDos());
        pool.submit(new TareaUno());
        pool.submit(new TareaDos());   // este ya llena los 3 hilos
        pool.submit(new TareaUno());   // se queda en cola

        pool.shutdown();                                    // no acepta
mas tareas
        pool.awaitTermination(30, TimeUnit.SECONDS);     // esperar
(como join)
        // pool.shutdownNow();                              // interrumpe
las que sigan
    }
}

public class TareaUno implements Runnable {
    @Override
    public void run() {
        System.out.println("Probando Executors - " +
Thread.currentThread().getName());
    }
}

public class TareaDos implements Runnable {
    @Override
    public void run() {
        while (true) {                // no termina: bloquea un hilo
del pool
            System.out.println("Probando Executors 2");
            try { Thread.sleep(10000); } catch (InterruptedException
e) { return; }
        }
    }
}
```

Método	Qué crea
`Executors.newFixedThreadPool(n)`	Pool con n hilos fijos
`Executors.newSingleThreadExecutor()`	Un solo hilo, tareas en orden
`Executors.newCachedThreadPool()`	Crea hilos según haga falta
`Executors.newScheduledThreadPool(`	Pool para tareas con retraso o periódicas

Método	Qué crea
`n)`	
`Executors.newSingleThreadScheduledExecutor()`	Un hilo para tareas programadas

12. Exchanger: intercambiar objetos entre dos hilos [EXCHANGER]

`exchange(obj)` entrega `obj` y devuelve el que entrega el otro hilo. El primero que llega se bloquea hasta que llega el otro. Con `exchange(obj, tiempo, unidad)` se pone un timeout. Aquí el productor rellena cadenas de 10 caracteres durante 1 s; para parar, envía una cadena vacía.

(En tus apuntes falta la clase `HiloProductor`; aquí está completa.)

```
import java.util.concurrent.Exchanger;

public class MainExchanger {
    public static void main(String[] args) {
        Exchanger<String> intercambiador = new Exchanger<>();
        HiloProductor productor = new HiloProductor(intercambiador);
        productor.start();
        new HiloConsumidor(intercambiador).start();

        try {
            Thread.sleep(1000);       // dejarles intercambiar durante
1 segundo
        } catch (InterruptedException ex) {
            System.out.println(ex.getMessage());
        }
        productor.parada();           // el productor enviara ""
(senal de fin) al consumidor
    }
}

import java.util.concurrent.Exchanger;

public class HiloProductor extends Thread {
    private final Exchanger<String> intercambiador;
    private volatile boolean activo = true;   // volatile: el cambio
lo ve este hilo enseguida

    HiloProductor(Exchanger<String> intercambiador) {
        this.intercambiador = intercambiador;
    }

    public void parada() { activo = false; }

    @Override
    public void run() {
```

```
        try {
            while (activo) {
                StringBuilder sb = new StringBuilder();
                for (int i = 0; i < 10; i++) {
                    sb.append((char) ('a' + (int) (Math.random() *
26)));
                }
                intercambiador.exchange(sb.toString());   // entrega
la cadena, recibe ""
            }
            intercambiador.exchange("");                  // senal de
parada
        } catch (InterruptedException ex) {
            System.out.println(ex.getMessage());
        }
    }
}

import java.util.concurrent.Exchanger;

public class HiloConsumidor extends Thread {
    private final Exchanger<String> intercambiador;
    private String cadena;

    HiloConsumidor(Exchanger<String> intercambiador) {
        this.intercambiador = intercambiador;
    }

    @Override
    public void run() {
        while (cadena == null || cadena.length() > 0) {   // hasta
recibir ""
            try {
                cadena = intercambiador.exchange("");      // entrega
"" y recibe la llena
                if (cadena.length() > 0) {
                    System.out.println("Consumidor recibe " + cadena
                            + ", mientras el productor genera la
siguiente...");
                }
            } catch (InterruptedException ex) {
                System.out.println(ex.getMessage());
            }
        }
    }
}
```

13. Errores típicos de examen [ERRORES]

Síntoma	Causa	Solución
Los hilos se ejecutan uno detrás de otro	Llamaste a `run()` en vez de `start()`	Usar `start()`
`IllegalThreadStateException`	`start()` dos veces sobre el mismo Thread	Crear un `new Thread` nuevo
El contador da menos de lo esperado	Sección crítica sin proteger	`synchronized` sobre el objeto compartido
`synchronized` no funciona	Cada hilo bloquea su propio `this`	Bloquear el objeto compartido o uno `static`
`IllegalMonitorStateException`	`wait()/notify()` fuera de `synchronized`	Meterlos en un método/bloque `synchronized` del mismo objeto
Un consumidor saca de una lista vacía	Condición con `if` en vez de `while`	`while (condicion) wait();`
El programa no termina	Timer sin `cancel()`, Executor sin `shutdown()`, hilo con bucle infinito	`cancel()`, `shutdown()`, `interrupt()`
El main imprime antes de que acaben los hilos	Falta `join()`	`join()` de cada hilo antes de imprimir
El hilo no para con `interrupt()`	Se captura `InterruptedException` y se sigue en el bucle	`break/return` en el catch
Programa congelado sin error	Interbloqueo	Pedir cerrojos siempre en el mismo orden
Semáforo se queda sin permisos	`release()` no se ejecuta si hay excepción	`release()` en `finally`

Esqueleto rápido: N hilos sobre un recurso compartido

```
public class Main {
    public static void main(String[] args) throws InterruptedException
{
        Recurso r = new Recurso();                          // objeto
compartido
        Thread[] hilos = new Thread[5];                     // <-
CAMBIAR numero
        for (int i = 0; i < hilos.length; i++) {
            final int id = i;
            hilos[i] = new Thread(() -> {
                for (int j = 0; j < 1000; j++) {            // <-
CAMBIAR trabajo
                    r.operar(id);
                }
```

```
            }, "Hilo-" + i);
            hilos[i].start();
        }
        for (Thread h : hilos) h.join();
        System.out.println("Resultado: " + r.getValor());
    }
}

class Recurso {
    private int valor = 0;
    public synchronized void operar(int id) { valor++; }
    public synchronized int getValor() { return valor; }
}
```